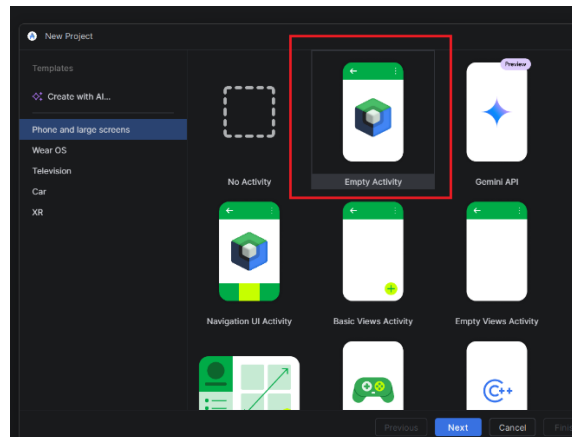




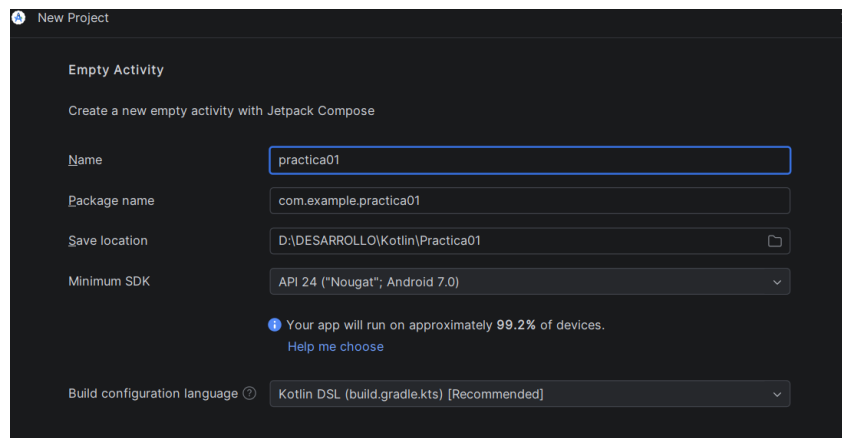
PRACTICA 01 — HOLA MUNDO



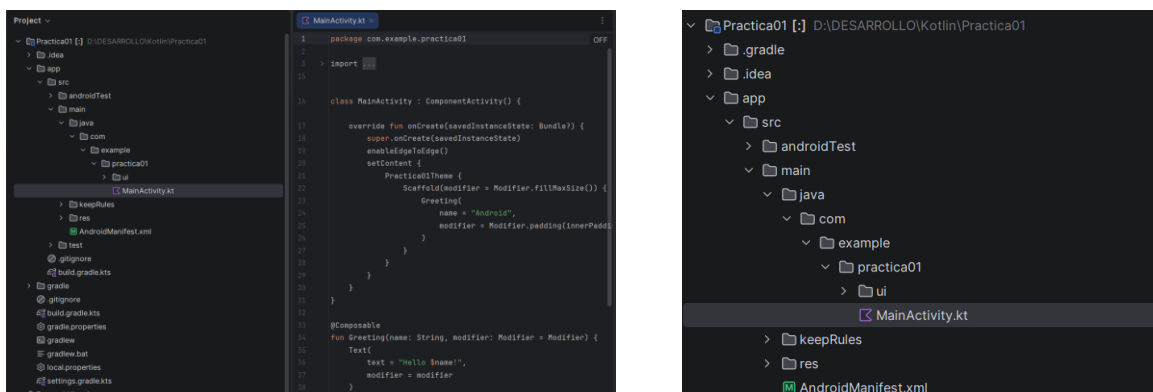
Paso 1: Iniciaremos *Android Studio* y seleccionamos **Empty Activity**



Paso 2: Ponemos el nombre del proyecto como **practica01**, así como la ruta donde estará mi proyecto y versión de *Android* la cual estaré manejando.



Paso 3: Tendremos la estructura que *Android Studio* nos genera de forma automática. La cual analizaremos en específico.



- La carpeta **.gradle** contiene los archivos temporales de **Gradle** que se utilizan para poder compilar el proyecto, acá se guarda el caché y nos ayuda de cierta manera en acelerar procesos de **build**, además de guardar todas las dependencias necesarias del proyecto.



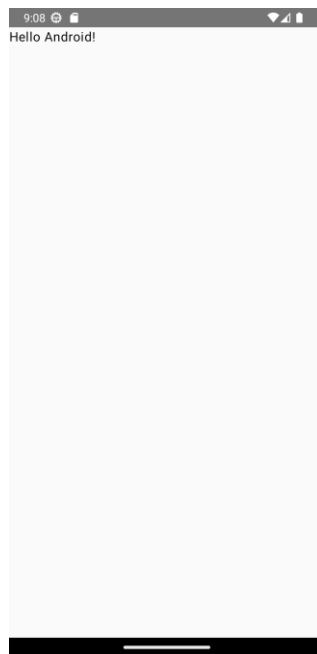
PRACTICA 01 — HOLA MUNDO



Por lo general, *Android Studio* lo gestiona automáticamente y por eso **NO** debemos de modificarlo.

- La carpeta **.idea** contiene la configuración interna de *Android Studio*, es decir, acá guarda cosas referente al tema del proyecto, configuración del IDE y ciertos ajustes necesarios para la ejecución del proyecto. Por lo general, *Android Studio* lo gestiona automáticamente y por eso **NO** debemos de modificarlo
- La carpeta **app** es la más importante. Ya que representa la aplicación Android que estamos construyendo. Acá va estar nuestro código en Kotlin, recursos, Manifest y ciertas configuraciones del módulo.
- La carpeta **main\java** se encuentran el conjunto de subcarpetas que vienen del **Package Name** que pusimos a nuestro proyecto. Teniendo como archivo especial el **MainActivity.kt**. El cual es donde esta todo el código Kotlin de nuestra aplicación.

Paso 4: Si ejecutamos la aplicación y esperamos que el Gradle compile nuestra aplicación por default en Android se verá de la siguiente manera.



Bien, ahora toca explicar el código para ir conociendo un poco como es que se compone la aplicación por default en Android.

Nota: No pondré el código completo, ya que cuando se genera **Empty Activity** *Android Studio* me lo genera en automático. Solo explicaremos que hace cada bloque de código para ir conociendo el código *Kotlin* que nos genera de forma automática.

En un primer inicio tenemos la clase **MainActivity** que hereda de **ComponentActivity**. La cual un **Activity** es la pantalla de nuestra aplicación, la cual podemos tener muchas pantallas en nuestra aplicación. Luego tenemos **onCreate()** es el primer método que Android ejecuta cuando abre una



PRACTICA 01 — HOLA MUNDO

Activity. Lo antecede de un **override** el cual dice que esta reemplazando el comportamiento de un método que ya existe en **ComponentActivity**.

Luego aparece la línea **super.onCreate()** la cual hace un llamado a la implementación original de Android y en ciertos **Activity** es obligatorio hacerlo.

Después tenemos la línea **enableEdgeToEdge()** lo cual va a permitir que nuestra aplicación cubra toda la pantalla. Desde la barra del estado superior como la inferior. Este código de línea va depender del tipo de aplicación que vayamos hacer, aunque en las apps modernas su uso es de lo más normal.

Después tenemos la línea más importante del **JetPackCompose** y es **setContent{}.** Ya que básicamente todo lo que este dentro de las llaves va representar **UI** de mi aplicación. Anteriormente la UI se componían de archivos **XML** el cual hasta la llegada de **JetPackCompose** era algo obligatorio conocer. Ahora lo más utilizado es utilizar el método actualmente estaremos manejando.

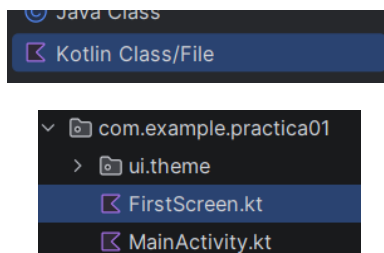
Las demás líneas de código de la clase **MainActivity** ya no la explicaremos, ya que posteriormente estaremos trabajando con ellos como es la cuestión de los temas, de los modificadores de comportamiento visual. Nos centraremos en las líneas de código que antes de su bloque de código esta **@Composable.**

Ya que al hacer un bloque de código por lo general una función y a este bloque le antecede la línea **@Composable** lo que se esta diciendo básicamente que la función pertenece a **JetPackCompose** y puede ser usado en la en el **setContent** y por lo consiguiente aparecer en la UI de nuestra aplicación.

Cuando aparezca la línea **@Preview** le dice al *Android Studio* “Muestrame una vista previa sin que ejecutes la aplicación”. A esta línea le pueden agregar ciertos modificadores, como es el caso del ejemplo que aparece **showBackground = true.** Que le dice al Android Studio previsualiza la función y agrégale un color de fondo. Por lo general, el definido por los temas de Android.

Bien ya conociendo las partes esenciales de archivo **MainActivity.kt** empezaremos con nuestra practica.

Paso 5: Crearemos un nuevo **archivo** en la raíz de nuestro proyecto. Es decir, en este momento lo tendremos en la raíz donde se encuentra el archivo **MainActivity.kt** y a este nuevo archivo le pondremos de nombre **FirstScreen.kt.**

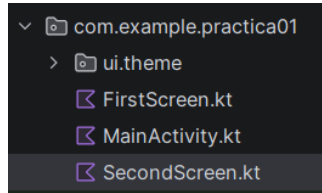




PRACTICA 01 — HOLA MUNDO



Paso 6: También crearemos otro archivo ahí mismo en la raíz. Solo que a este le pondremos el nombre de **SecondActivity.tk**.



Paso 7: Borraremos del archivo **MainActivity.tk** las líneas de código que se encuentran dentro del **setContent** que están en la clase **MainActivity** y las líneas de código que están por fuera de esa clase. Es decir, todo aquello que aparezcan las líneas **@Composable** y **@Preview**.

Paso 8: Agregamos las siguientes líneas de código en **FirstScreen.tk** después de la línea 1 que trae nuestro archivo.

```
FirstScreen.tk

@Composable
fun FirstScreen() {
    var textoIngresado by remember { mutableStateOf("") }
    Scaffold(
        containerColor = Color(0xFFE3F2FD)
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .padding(16.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            OutlinedTextField(
                value = textoIngresado,
                onValueChange = { textoIngresado = it },
                label = { Text("Escribe tu texto") },
                modifier = Modifier.fillMaxWidth()
            )

            Spacer(modifier = Modifier.height(20.dp))

            Button(
                onClick = { }
            ) {
                Text("Ir a SecondScreen")
            }
        }
    }
}
```

Donde lo que hacen las siguientes líneas de código es que agregamos esta la declaración de la variable **textoIngresado**. La cual la estamos inicializando vacía en un inicio pero es importante mencionar la palabra **remember** y su importancia al usar **JetPackCompose** ya que siempre está reconstruyendo la interfaz y si no usáramos dicha palabra el valor volvería a ser vacío cada vez que



PRACTICA 01 — HOLA MUNDO



la pantalla se redibujara. Además **mutableStateOf** hace que nuestra variable se convierta en un estado *observable*, es decir, **Compose** estaría al pendiente de que si hay un cambio está se mandará a redibujar.

Después tenemos **Scaffold**, y esta es una estructura base de una pantalla. La cual contiene elementos como barra superior (*ToAppBar*), menú lateral, botón flotante y el cuerpo de la aplicación. Tiene ciertos modificadores están el color de fondo (**containerColor** e **innerPadding**). Dentro del mismo tenemos Column lo cual ira organizando elementos de forma vertical. Como lo son **OutlinedTextField**, **Spacer** y **Button**. Algunos componentes **Composable**, como **Button**, requieren ciertos parámetros obligatorios para funcionar correctamente. Uno de ellos es **onClick**, que define la acción que se ejecutará cuando el usuario presione el botón. Por el momento no hemos asignado ninguna acción a este evento, por lo que el bloque **onClick** se encuentra vacío. Únicamente hemos definido el contenido del botón mediante el componente **Text**, que muestra el texto visible al usuario.

Paso 9: Ahora nos dirigimos al archivo **MainActivity.tk** y en las partes del código **setContent** agregaremos las siguientes líneas de código.

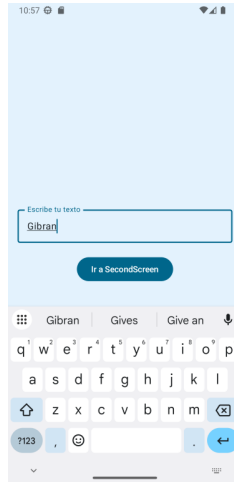
```
MainActivity.tk

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Practica01Theme {
                FirstScreen()
            }
        }
    }
}
```

Si observas bien solo agregamos la función **Composable** que se encuentra en el archivo **FirstScreen**. Lo cual si ejecutamos nuestra aplicación **MainActivity** se va redirigir a lo creado en **FirstScreen.tk**. El resultado sería lo siguiente:



PRACTICA 01 — HOLA MUNDO



Paso 10: En el archivo **SecondScreen.kt** trabajaremos de forma muy parecida a como lo hicimos en **FirstScreen.kt**. Sin embargo, esta pantalla tendrá una responsabilidad diferente: recibir y mostrar un mensaje proveniente de la primera pantalla. Por esta razón, ya no necesitaremos componentes de entrada de datos como **OutlinedTextField** ni componentes interactivos como **Button**. En su lugar, utilizaremos dos componentes **Text**, que nos permitirán presentar información al usuario de manera sencilla y clara. También vamos a aprovechar de cambiar el color de fondo para que veamos de forma más visible que son dos **Activity** distintos. El código quedaría así.



PRACTICA 01 – HOLA MUNDO



```
SecondScreen.tk

@Composable
fun SecondScreen(mensaje: String) {
    Scaffold(
        containerColor = Color(0xFF3E5F5)
    ) { innerPadding →
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .padding(16.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "Texto recibido:",
                fontSize = 18.sp
            )

            Spacer(modifier = Modifier.height(8.dp))

            Text(
                text = mensaje,
                fontSize = 24.sp
            )
        }
    }
}
```

Hasta este momento ya contamos con las dos pantallas que formarán parte de nuestra práctica. Sin embargo, todavía falta implementar uno de los elementos más importantes: la navegación entre pantallas.

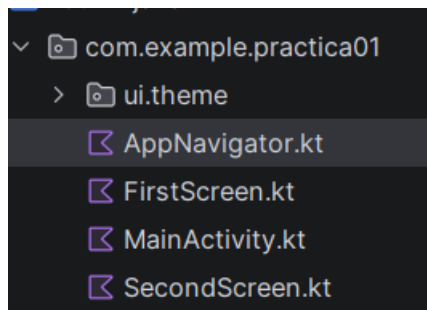
Para lograrlo, crearemos un **NavHost** o sistema de navegación, que será el encargado de gestionar a qué pantalla debe dirigirse la aplicación y cómo poder regresar a la pantalla anterior cuando sea necesario.

Además, nuestra implementación aún no está completa. Aunque ya tenemos las pantallas **FirstScreen** y **SecondScreen**, todavía debemos enviar el texto capturado en **FirstScreen** hacia **SecondScreen**, para que este pueda ser recibido y mostrado al usuario.

Paso 11: Crearemos un nuevo **archivo** en la raíz de nuestro proyecto a este nuevo archivo le tendrá el nombre **AppNavigator.kt**. Este será el encargado de controlar el flujo de la aplicación.



PRACTICA 01 – HOLA MUNDO



Antes de agregar líneas de código a **AppNavigator.kt**, necesitamos realizar unos ajustes a nuestro archivo **FirstScreen.kt**. El primer punto en realizar es el pasar el parámetro de navegación y después que el **Button** realice una acción que sería “Lanzar” el **SecondScreen**. Las líneas modificadas quedarían así:



A continuación, trabajaremos en el archivo **AppNavigator**, el cual será el encargado de gestionar la navegación entre las distintas pantallas de la aplicación. Aquí definiremos cuál será la pantalla que se mostrará inicialmente cuando el usuario ejecute la aplicación.

En nuestro caso, comenzaremos mostrando **FirstScreen**; sin embargo, durante el proceso de desarrollo podemos cambiar esta configuración según nuestras necesidades. Por ejemplo, si estamos trabajando en **SecondScreen** y queremos revisar rápidamente su comportamiento o apariencia, podemos establecerla temporalmente como pantalla inicial. Esto nos permite realizar pruebas de forma más rápida y eficiente sin necesidad de navegar por todas las pantallas cada vez que ejecutamos la aplicación.

El código quedaría así:



PRACTICA 01 – HOLA MUNDO



```
AppNavigator.tk

@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = "first_screen"
    ) {
        // Pantalla 1 - FirstScreen
        composable("first_screen") {
            FirstScreen(onNavigateToSecondScreen = { texto →
                navController.navigate("second_screen/$texto")
            })
        }

        // Pantalla 2 con argumento - SecondScreen
        composable(
            route = "second_screen/{texto}",
            arguments = listOf(navArgument("texto") { type = NavType.StringType })
        ) { backStackEntry →
            val textoRecibido = backStackEntry.arguments?.getString("texto") ?: ""
            SecondScreen(mensaje = textoRecibido)
        }
    }
}
```

Analizando con más detalle la configuración de la segunda ruta, podemos observar el uso de **arguments = listOf(...)**. La función **listOf()** crea una lista que contendrá todos los parámetros que la pantalla puede recibir durante la navegación. En este caso, únicamente se define un argumento llamado **texto** mediante **navArgument("texto")**, indicando además que su tipo será *String* con **NavType.StringType**. Gracias a esta configuración, **Navigation Compose** sabe que la ruta espera recibir una cadena de texto y puede validar el dato antes de entregarlo a la pantalla de destino.

Cuando se produce la navegación, los argumentos enviados se almacenan temporalmente en un objeto denominado **NavBackStackEntry**, representado en el código mediante la variable **backStackEntry**. Este objeto forma parte del **Back Stack**, una especie de historial de navegación que Android mantiene internamente para recordar las pantallas visitadas y permitir que el usuario regrese a ellas utilizando el botón **Atrás**. Además de conservar información sobre la pantalla actual, el **backStackEntry** también almacena los argumentos recibidos durante la navegación.

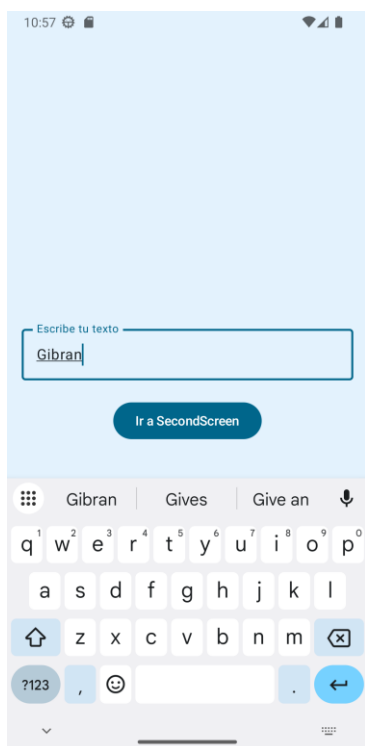


PRACTICA 01 — HOLA MUNDO

Por esta razón, para recuperar el mensaje enviado desde **FirstScreen**, utilizamos la expresión **backStackEntry.arguments?.getString("texto")**. Aquí se accede a la colección de argumentos asociados a la ruta actual y se obtiene el valor correspondiente al argumento llamado texto. Finalmente, el operador **?: ""**, conocido en Kotlin como **Operador Elvis**, proporciona una cadena vacía como valor predeterminado en caso de que el argumento no exista o sea nulo, evitando posibles errores durante la ejecución de la aplicación.

En resumen, **navArgument()** define qué datos puede recibir una pantalla, **listOf()** agrupa dichos argumentos, **backStackEntry** almacena la información de navegación de la pantalla actual y **arguments.getString()** permite recuperar los valores enviados desde una pantalla anterior para utilizarlos dentro del **composable** de destino.

Paso 12: Ejecutamos nuestra aplicación en Android.



Reto:

Una vez que hayas completado de practica, ahora deberás integrar en el **SecondScreen** justo en el **TopAppBar** un botón de retorno. Ya que no pusimos un botón para hacer esto, solo podemos regresar al **FirstScreen** con el botón físico del dispositivo o emulador.

